Guide d'explication

POO : Personnage, Heros, Monstre (RPG) — comment chaque fonction a été codée, et pourquoi

Ce document reprend **votre code** et les **consignes** de l'exercice. Pour chaque fonction, on suit toujours le même fil en cinq étapes, de la lecture de l'énoncé jusqu'à la justification du code :

1 LA CONSIGNE	Ce que dit l'énoncé, mot pour mot ou presque.
2 COMPRENDRE	Ce qu'on nous demande vraiment : entrées, sorties, cas particuliers, pièges.
3 S'Y PRENDRE	Le plan en étapes, avant d'écrire la moindre ligne.
4 RÉALISÉ	Votre code, ligne par ligne, avec ce qu'il fait.
5 POURQUOI	Les raisons des choix : ce qu'on gagne, ce qu'on évite.

Les trois sortes de blocs de code

Votre code

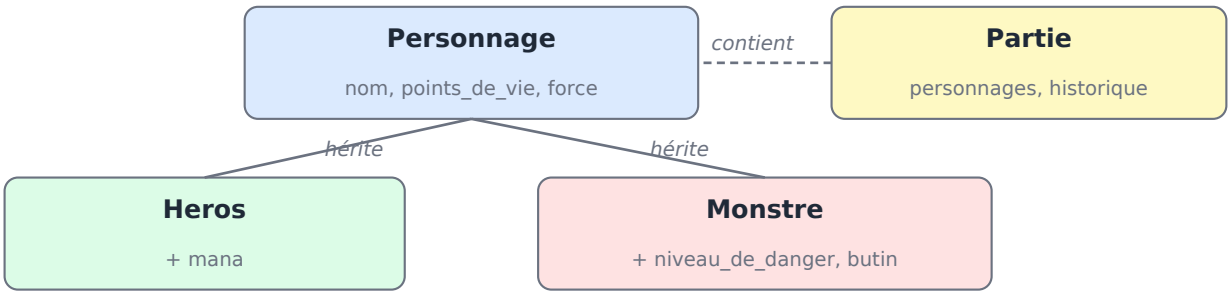
```
# Bloc gris : votre code, tel que vous l'avez écrit
x = 1
```

Version corrigée

```
# Bloc vert : version corrigée ou améliorée
x = 1
```

Encadré orange : un écart entre votre code et la consigne (ou un comportement inattendu). Chaque écart a été **vérifié en exécutant le code** ; la liste complète est résumée en fin de document.

Vue d'ensemble : ce qu'on construit



Partie	Ce qu'on construit	Notions clés
A	Terminer la classe **Personnage** : subir_degats, attaquer, __repr__	méthodes, *return* vs *print*, réutilisation
B	Les classes **Heros** et **Monstre**	héritage, super(), redéfinition, polymorphisme
C	Sauvegarde / chargement d'un personnage en **.json**	dictionnaires, module json, fichiers
D	La classe **Partie** : combats, historique, sauvegarde	délégation, composition de classes

Partie A — Compléter la classe Personnage

Objectif : terminer trois méthodes (subir_degats, attaquer, __repr__) d'une classe déjà commencée.

A.0 Le socle fourni : __init__ et est_vivant

1 LA CONSIGNE

Fournis par l'énoncé, rien à écrire : le constructeur range *nom*, *points_de_vie* et *force* ; `est_vivant` dit si le personnage a encore des points de vie.

2 COMPRENDRE LA CONSIGNE

Avant de coder, il faut lire ces deux méthodes : elles donnent les **noms d'attributs** à utiliser partout ensuite (`self.nom`, `self.points_de_vie`, `self.force`) et un test tout prêt pour savoir si quelqu'un est encore en vie.

3 COMMENT S'Y PRENDRE

1. Repérer les attributs de l'objet.
2. Repérer ce que renvoie `est_vivant` : un booléen (`True` / `False`).

4 COMMENT C'EST RÉALISÉ

Code fourni

```
class Personnage:
    def __init__(self, nom, points_de_vie, force):
        self.nom = nom
        self.points_de_vie = points_de_vie
        self.force = force

    def est_vivant(self):
        return self.points_de_vie > 0
```

5 POURQUOI COMME ÇA

Sans `self.`, une valeur disparaîtrait à la fin du constructeur : `self.nom = nom` la fait **vivre dans l'objet**. `est_vivant` renvoie directement le résultat de la comparaison : pas besoin de `if ... return True else return False`. Tout le programme se fiera à ce test, y compris la partie D.

A.1 subir_degats(self, degats)

1 LA CONSIGNE

« Diminue `self.points_de_vie` de `degats`, **sans jamais descendre sous 0**. »

2 COMPRENDRE LA CONSIGNE

Cette méthode **modifie l'objet** : elle ne renvoie rien. Le piège est la contrainte « jamais négatif » : un personnage à 5 PV qui subit 8 dégâts doit finir à **0**, pas à -3. Sinon `est_vivant` resterait correct (-3 n'est pas > 0) mais l'affichage montrerait « -3 PV », ce qui n'a aucun sens dans le jeu.

3 COMMENT S'Y PRENDRE

1. Soustraire les dégâts aux points de vie.
2. Vérifier si le résultat est négatif.
3. Si oui, le ramener à 0.

4 COMMENT C'EST RÉALISÉ

Votre code

```
def subir_degats(self, degats):
    self.points_de_vie -= degats
    if self.points_de_vie < 0:
        self.points_de_vie = 0
    self.points_de_vie = self.points_de_vie     # <- cette ligne ne sert à rien
```

Ligne par ligne : `-=` retire les dégâts ; le `if` « plafonne » à 0 ; la dernière ligne réaffecte la valeur à elle-même, elle n'a **aucun effet** et peut être supprimée. Une écriture plus courte existe :

Version équivalente, plus courte

```
def subir_degats(self, degats):
    self.points_de_vie = max(0, self.points_de_vie - degats)
```

5 POURQUOI COMME ÇA

On passe par `self.points_de_vie` car c'est l'état de l'objet qui doit changer (pas de `return` : c'est un *effet*, pas un *résultat*). Surtout, cette règle « jamais sous 0 » est écrite **une seule fois** ici : `attaquer`, `competence_speciale` et la classe `Monstre` passeront toutes par `subir_degats` et en profiteront sans recopier le test.

A.2 attaquer(self, cible)

1 LA CONSIGNE

« Applique `self.force` en dégâts à `cible` (via `subir_degats`), puis **RETOURNE** (return, pas print) un message du type : *Aldric attaque Gobelin et lui inflige 8 dégâts (Gobelin : 12 PV restants)*. »

2 COMPRENDRE LA CONSIGNE

Deux choses distinctes sont demandées : un **effet** (la cible perd des PV) et un **résultat** (un texte renvoyé). La différence `return` / `print` est essentielle : `print` affiche à l'écran puis la fonction renvoie `None` ; `return` donne le texte à celui qui appelle. En partie D, `Partie.combat` doit *ranger* ce texte dans une liste : c'est impossible avec un print.

Piège de lecture : dans le message, les PV restants sont ceux **après** l'attaque (12 = 20 - 8).

3 COMMENT S'Y PRENDRE

1. Calculer les dégâts : ici, simplement `self.force`.
2. Appeler `cible.subir_degats(degats)` (on réutilise, on ne touche pas aux PV directement).
3. Construire le message **après** l'attaque, pour que les PV soient à jour.
4. Le retourner avec `return`.

4 COMMENT C'EST RÉALISÉ

Votre code (retours à la ligne ajoutés pour tenir dans la page)

```
def attaquer(self, cible: "Personnage"):
    if not cible.est_vivant():
        return f"{cible.nom} est deja vaincu"
    degats = self.force
    cible.subir_degats(degats)
    return (f"{self.nom} attaque {cible.nom} et lui inflige "
            f"{degats}dégats ({cible.nom}: {cible.points_de_vie} PV restants)")
```

Les quatre étapes du plan sont bien là. Vous avez ajouté de vous-même une **garde** (« déjà vaincu ») : bonne idée, frapper un personnage mort n'aurait pas de sens. L'annotation `cible: "Personnage"` est un simple indice de type, sans effet à l'exécution.

Point d'attention. Il manque l'espace dans « *{degats}dégats* » (affiche « 8dégats ») et le format s'écarte de l'exemple : « *(Gobelin : 12 PV restants)* » avec une espace avant les deux-points. Les f-strings reproduisent le texte **caractère pour caractère**, espaces comprises.

Version corrigée de la ligne de retour

```
return (f"{self.nom} attaque {cible.nom} et lui inflige {degats} dégâts "
        f"({cible.nom} : {cible.points_de_vie} PV restants)")
```

5 POURQUOI COMME ÇA

Réutiliser `subir_degats` plutôt que faire `cible.points_de_vie -= degats` : si la règle du plancher à 0 évolue, un seul endroit à modifier. **Retourner** plutôt qu'afficher : la même méthode sert à l'affichage (`print(...)`), à l'historique (partie D) et pourrait servir à des tests. Le f-string est la façon la plus lisible d'insérer des variables dans un texte.

A.3 __repr__(self)

1 LA CONSIGNE

« Retourne une chaîne du type *Aldric (20 PV)*. »

2 COMPRENDRE LA CONSIGNE

`__repr__` est une méthode « spéciale » : Python l'appelle tout seul quand on écrit `print(objet)`. Sans elle, `print(gobelin)` afficherait quelque chose d'illisible comme `<__main__.Personnage object at 0x7f...>`. Elle doit **retourner** une chaîne, pas l'afficher. Le format demandé est : nom, espace, PV entre parenthèses.

3 COMMENT S'Y PRENDRE

1. Récupérer `self.nom` et `self.points_de_vie`.
2. Les assembler dans le format demandé avec un f-string.
3. Retourner la chaîne.

4 COMMENT C'EST RÉALISÉ

Votre code

```
def __repr__(self):
    return (f"{self.nom} : {self.points_de_vie} PV")
```

Le mécanisme est correct (f-string, `return`), mais le format produit *« Aldric : 20 PV »* au lieu de *« Aldric (20 PV) »*.

Version corrigée

```
def __repr__(self):
    return f"{self.nom} ({self.points_de_vie} PV)"
```

5 POURQUOI COMME ÇA

Écrite dans la classe de base, cette méthode est **héritée** : `print(lancelot_recharge)` (partie C) fonctionnera sans rien ajouter à `Heros`. Respecter le format exact compte : un test automatique comparerait la chaîne telle quelle.

Partie B — Héritage : Heros et Monstre

Objectif : créer deux classes filles qui réutilisent Personnage et y ajoutent leurs particularités.

B.1 Heros.__init__(self, nom, points_de_vie, force, mana)

1 LA CONSIGNE

« Heros a un attribut supplémentaire `mana` (entier). Le constructeur doit appeler `super().__init__(nom, points_de_vie, force)` puis définir ses attributs propres. »

2 COMPRENDRE LA CONSIGNE

Un héros **est** un personnage (il a nom, PV, force) **avec en plus** du mana. L'héritage sert exactement à ne pas recopier ce qui existe déjà. Le constructeur du héros doit donc recevoir les trois valeurs du parent *plus* `mana`.

3 COMMENT S'Y PRENDRE

1. Déclarer `class Heros(Personnage)` : la parenthèse indique le parent.
2. Écrire un `__init__` dont les paramètres sont ceux du parent + `mana`.
3. Déléguer au parent avec `super().__init__(...)`.
4. Ajouter `self.mana = mana`.

4 COMMENT C'EST RÉALISÉ

Votre code

```
class Heros(Personnage):

    def __init__(self, nom, points_de_vie, force, mana):
        super().__init__(nom, points_de_vie, force)
        self.mana = mana
```

5 POURQUOI COMME ÇA

`super().__init__` exécute le constructeur de `Personnage` : les trois attributs communs sont créés au même endroit pour toutes les classes. Si demain `Personnage` reçoit un nouvel attribut, `Heros` et `Monstre` en bénéficient automatiquement.

B.2 Heros.competence_speciale(self, cible)

1 LA CONSIGNE

« Coûte 10 points de mana et inflige le **double** de `self.force` à `cible` (en réutilisant `subir_degats`) ; si le héros n'a pas assez de mana, la méthode **ne fait rien** et retourne un message l'indiquant. Dans tous les cas, elle **retourne** un message. »

2 COMPRENDRE LA CONSIGNE

Deux scénarios à bien séparer :

- **Assez de mana** (au moins 10) : dégâts = force × 2, mana diminue de 10, message de succès.
- **Pas assez de mana** (moins de 10) : rien ne change (ni dégâts, ni perte de mana), message d'échec.

Le test fourni le confirme : mana 25 → 15 → 5, puis le 3e appel échoue. Notez qu'avec **exactement** 10 de mana, la compétence doit être possible.

3 COMMENT S'Y PRENDRE

1. **Vérifier d'abord** : si `self.mana < 10`, retourner tout de suite le message d'échec.
2. Sinon, calculer `degats = self.force * 2`.
3. Appeler `cible.subir_degats(degats)`.
4. Retirer 10 au mana.
5. Retourner le message de succès.

4 COMMENT C'EST RÉALISÉ

Votre code (retours à la ligne ajoutés)

```
def competence_speciale(self, cible: "Personnage"):
    degats = self.force
    cible.subir_degats(degats * 2)
    self.mana -= 10
    if self.mana < 10:
        print("Plus de points de mana")
        self.mana = 0
    else:
        self.mana = self.mana
    return (f"{self.nom} attaque {cible.nom} et lui inflige {degats * 2} degats "
            f"et lui reste {self.mana} de mana "
            f"({cible.nom}: {cible.points_de_vie} PV restants)")
```

Les bons réflexes sont là : `subir_degats` réutilisée, double de la force, 10 de mana retirés, message retourné. Mais l'**ordre** des opérations ne correspond pas au raisonnement de l'étape 3 : le code frappe *d'abord*, teste le mana *ensuite*.

> **Points d'attention (comportement réellement observé en exécutant votre code avec le test de l'énoncé).**
> • Le test `self.mana < 10` porte sur le mana **après** avoir retiré 10. Au 2e appel (mana 15 → 5), le code croit à tort que le mana manque : il affiche « Plus de points de mana » et remet le mana à **0** au lieu de 5.
> • Au 3e appel, les dégâts sont **quand même infligés** (la cible perd 20 PV) alors que la consigne dit « ne fait rien ».
> • Le message d'échec est un `print` ; la consigne veut un `return`. Le `else: self.mana = self.mana` est sans effet.

Version corrigée

```
def competence_speciale(self, cible):
    if self.mana < 10:                                  # 1) on vérifie AVANT d'agir
        return (f"{self.nom} n'a pas assez de mana "
                f"({self.mana} < 10) : compétence impossible")
    degats = self.force * 2                             # 2) on agit
    cible.subir_degats(degats)
    self.mana -= 10
    return (f"{self.nom} utilise sa compétence spéciale sur {cible.nom} "
            f"et lui inflige {degats} dégâts "
            f"({cible.nom} : {cible.points_de_vie} PV restants, "
            f"mana de {self.nom} : {self.mana})")
```

Résultat avec le test de l'énoncé (mana 25 → 15 → 5 → échec) :

```
Lancelot utilise sa compétence spéciale sur Gobelin et lui inflige 20 dégâts (... mana : 15)
Lancelot utilise sa compétence spéciale sur Gobelin et lui inflige 20 dégâts (... mana : 5)
Lancelot n'a pas assez de mana (5 < 10) : compétence impossible
```

5 POURQUOI COMME ÇA

Le schéma « **vérifier, puis agir** » (clause de garde) évite d'avoir à annuler une action déjà faite, et rend le cas « impossible » explicite. Tester `< 10` *avant* de soustraire garantit que 10 de mana pile suffisent. Retourner le message dans les deux cas suit le principe posé en A.2 : c'est l'appelant qui décide d'afficher ou d'archiver.

B.3 Monstre.__init__(self, nom, points_de_vie, force, niveau_de_danger, butin)

1 LA CONSIGNE

« Monstre a deux attributs supplémentaires : `niveau_de_danger` (entier) et `butin` (chaîne, ex. « 10 pièces d'or »). »

2 COMPRENDRE LA CONSIGNE

Exactement le même mécanisme que pour `Heros`, avec **deux** attributs de plus au lieu d'un.

3 COMMENT S'Y PRENDRE

1. Paramètres du parent + `niveau_de_danger` + `butin`.
2. `super().__init__(nom, points_de_vie, force)`.
3. Deux affectations `self.…`.

4 COMMENT C'EST RÉALISÉ

Votre code

```
class Monstre(Personnage):

    def __init__(self, nom, points_de_vie, force, niveau_de_danger, butin):
        super().__init__(nom, points_de_vie, force)
        self.niveau_de_danger = niveau_de_danger
        self.butin = butin
```

5 POURQUOI COMME ÇA

Même raison qu'en B.1 : on ne recopie pas ce que le parent sait déjà faire. Le test `Monstre("Gobelin", 20, 5, niveau_de_danger=2, butin="...")` fonctionne parce que les noms de paramètres correspondent exactement à ceux de l'énoncé.

B.4 Monstre.attaquer(self, cible) — redéfinition (override)

1 LA CONSIGNE

« Redéfinissez (overridez) `attaquer` : un monstre inflige `self.force * self.niveau_de_danger` en dégâts au lieu de `self.force` seul (le reste du comportement — message retourné, appel à `subir_degats` — ne change pas). »

2 COMPRENDRE LA CONSIGNE

Redéfinir = écrire, dans la classe fille, une méthode **du même nom et de la même forme** que celle du parent, avec un comportement différent. Seule la formule des dégâts change : force × niveau de danger. Avec le test : 5 × 2 = 10 dégâts sur Lancelot (40 → 30 PV).

3 COMMENT S'Y PRENDRE

1. Copier la structure de `Personnage.attaquer`.
2. Remplacer la ligne des dégâts par `self.force * self.niveau_de_danger`.
3. Garder `subir_degats` et le `return` d'un message.

4 COMMENT C'EST RÉALISÉ

Votre code

```
def attaquer(self, cible: "Personnage"):
    degats = self.force * self.niveau_de_danger
    cible.subir_degats(degats)
    return (f"{self.nom} attaque {cible.nom} et lui inflige "
            f"{degats}dégats ({cible.nom}: {cible.points_de_vie} PV restants)")
```

Conforme à la consigne. Deux petites remarques : l'espace manquante dans « {degats}dégats » (voir A.2), et le fait que cette version n'a **pas** la garde « déjà vaincu » que vous avez mise dans `Personnage.attaquer` : un monstre pourrait donc frapper un héros déjà à 0 PV. Pour être cohérent, ajoutez la même garde :

Version cohérente

```
def attaquer(self, cible):
    if not cible.est_vivant():
        return f"{cible.nom} est déjà vaincu"
    degats = self.force * self.niveau_de_danger
    cible.subir_degats(degats)
    return (f"{self.nom} attaque {cible.nom} et lui inflige {degats} dégâts "
            f"({cible.nom} : {cible.points_de_vie} PV restants)")
```

5 POURQUOI COMME ÇA

C'est le **polymorphisme** : `gobelin.attaquer(lancelot)` et `aldric.attaquer(gobelin)` s'écrivent pareil, mais Python choisit automatiquement la bonne version selon la classe réelle de l'objet. `Heros` ne redéfinit pas `attaquer` : il utilise celle de `Personnage` (héritage). Grâce à cela, la classe `Partie` (partie D) pourra faire combattre n'importe qui sans se demander « est-ce un héros ou un monstre ? ».

Partie C — Sauvegarder un personnage au format .json

Objectif : transformer un objet en texte JSON, l'écrire dans un fichier, puis reconstruire l'objet.

C.1 to_dict(self) dans Personnage, Heros et Monstre

1 LA CONSIGNE

« Ajoutez à chaque classe une méthode `to_dict` qui retourne un dictionnaire de ses attributs, plus une clé `"type"` contenant le nom de la classe. `Heros.to_dict()` et `Monstre.to_dict()` doivent réutiliser `super().to_dict()` et y ajouter leurs attributs propres, en remplaçant `"type"`. »

2 COMPRENDRE LA CONSIGNE

JSON ne sait stocker que des données simples (texte, nombres, listes, dictionnaires) : **pas un objet Python**. Il faut donc d'abord « aplatir » l'objet en dictionnaire. La clé `"type"` est indispensable : plus tard, en relisant le fichier, c'est elle qui dira s'il faut recréer un `Personnage`, un `Heros` ou un `Monstre`. Piège : `super().to_dict()` renvoie `"type": "Personnage"` ; il faut **écraser** cette valeur dans les classes filles.

3 COMMENT S'Y PRENDRE

1. **Personnage** : écrire le dictionnaire complet à la main (type + 3 attributs).
2. **Heros** : partir de `super().to_dict()`, remplacer `"type"` par `"Heros"`, ajouter `"mana"`.
3. **Monstre** : même chose avec `"niveau_de_danger"` et `"butin"`.

4 COMMENT C'EST RÉALISÉ

Votre code

```
# dans Personnage
def to_dict(self):
    return {
        "type": "Personnage",
        "nom": self.nom,
        "points_de_vie": self.points_de_vie,
        "force": self.force,
    }

# dans Heros
def to_dict(self):
    dico: dict = super().to_dict()      # récupère les 4 clés du parent
    dico["type"] = "Heros"              # écrase "Personnage"
    dico["mana"] = self.mana            # ajoute l'attribut propre
    return dico

# dans Monstre
def to_dict(self):
    dico: dict = super().to_dict()
    dico["type"] = "Monstre"
    dico["niveau_de_danger"] = self.niveau_de_danger
    dico["butin"] = self.butin
    return dico
```

Conforme à la consigne. L'annotation `dico: dict` est facultative mais inoffensive.

5 POURQUOI COMME ÇA

Avec `super().to_dict()`, on ne **recopie pas** les clés déjà connues du parent : un attribut ajouté à `Personnage` apparaîtra automatiquement dans les deux classes filles. Chaque classe sait décrire ses

propres attributs, c'est elle la mieux placée pour le faire.

C.2 sauvegarder(self, chemin) dans Personnage

1 LA CONSIGNE

« Écrit `self.to_dict()` dans le fichier `chemin` au format JSON. Pas besoin de la redéfinir dans Heros/Monstre : grâce au polymorphisme, `self.to_dict()` appellera la bonne version. »

2 COMPRENDRE LA CONSIGNE

Une seule méthode, écrite **une fois** dans la classe de base. Quand `self` est un `Heros`, `self.to_dict()` exécute la version de `Heros` (avec le mana) ; quand c'est un `Monstre`, celle de `Monstre`. L'énoncé donne aussi le modèle d'écriture : `open(..., "w", encoding="utf-8")` puis `json.dump`.

3 COMMENT S'Y PRENDRE

1. Ouvrir le fichier **en écriture** (mode `"w"`) avec `with`.
2. Appeler `json.dump(self.to_dict(), f, ...)`.
3. Laisser le `with` refermer le fichier.

4 COMMENT C'EST RÉALISÉ

Votre code

```
def sauvegarder(self, path):
    file = open(path, "a", encoding="utf-8")
    json.dump(self.to_dict(), file, ensure_ascii=False, indent=4)
```

Deux points d'attention.
• Mode `"a"` (*append* = ajouter à la fin) au lieu de `"w"` (écraser). La 1re sauvegarde est correcte, mais la 2e colle un deuxième objet JSON derrière le premier : le fichier n'est plus du JSON valide et `json.load` plante avec *« Extra data »* (testé). Pour une sauvegarde, on veut le **dernier état** du personnage, donc on remplace le contenu.
• Le fichier n'est jamais refermé : sans `close()` ni `with`, les données peuvent ne pas être réellement écrites sur le disque à temps.

Version corrigée

```
def sauvegarder(self, chemin):
    with open(chemin, "w", encoding="utf-8") as f:
        json.dump(self.to_dict(), f, ensure_ascii=False, indent=4)
```

5 POURQUOI COMME ÇA

`with` ferme le fichier automatiquement, même en cas d'erreur. `ensure_ascii=False` garde les accents lisibles (« pièces d'or » plutôt que « pi\u00e8ces »). `indent=4` rend le fichier lisible par un humain (c'est un choix de confort, pas une obligation).

C.3 charger_personnage(chemin) — fonction hors classe

1 LA CONSIGNE

« Écrivez une fonction indépendante (hors classe) `charger_personnage(chemin)` qui lit un fichier .json et reconstruit l'objet de la bonne classe (Personnage, Heros ou Monstre) à partir de la clé `"type"`. »

2 COMPRENDRE LA CONSIGNE

C'est l'opération **inverse** de `sauvegarder` : fichier → dictionnaire → objet. La fonction est **hors classe** parce qu'au moment de lire le fichier, on n'a pas encore d'objet pour appeler une méthode, et on ne

sait pas encore de quelle classe sera le résultat : c'est le contenu du fichier qui le dit.

3 COMMENT S'Y PRENDRE

1. Ouvrir le fichier en lecture et le charger avec `json.load` : on obtient un dictionnaire.
2. Lire `dico["type"]`.
3. Selon sa valeur, appeler le bon constructeur avec les bonnes clés (Heros a 4 paramètres, Monstre 5, Personnage 3).
4. Si le type est inconnu, signaler le problème.

4 COMMENT C'EST RÉALISÉ

Votre code (légèrement remis en forme)

```
def charger_personnage(path):
    file = open(path, "r", encoding="utf-8")
    dico = json.load(file)

    if dico["type"] == "Heros":
        return Heros(dico["nom"], dico["points_de_vie"], dico["force"], dico["mana"])
    elif dico["type"] == "Monstre":
        return Monstre(dico["nom"], dico["points_de_vie"], dico["force"],
                       dico["niveau_de_danger"], dico["butin"])
    elif dico["type"] == "Personnage":
        return Personnage(dico["nom"], dico["points_de_vie"], dico["force"])
    else:
        return f"Il n'existe pas de type de personnage de la sorte {dico['type']}"
```

La logique est la bonne : un `if/elif` par type, chaque constructeur appelé avec les clés correspondantes (celles posées par les `to_dict`).

> **À améliorer.** (1) Le fichier n'est pas refermé : utiliser `with`. (2) En cas de type inconnu, la fonction **retourne un texte** : l'appelant croirait avoir reçu un personnage et planterait plus loin, de façon incompréhensible. Une erreur doit être **levée** avec `raise`. (3) Cette même série de `if/elif` est recopiée dans `charger_partie` (bonus) : on la met dans une petite fonction réutilisable.

Version corrigée

```
def personnage_depuis_dict(dico):
    type_perso = dico["type"]
    if type_perso == "Heros":
        return Heros(dico["nom"], dico["points_de_vie"], dico["force"], dico["mana"])
    if type_perso == "Monstre":
        return Monstre(dico["nom"], dico["points_de_vie"], dico["force"],
                       dico["niveau_de_danger"], dico["butin"])
    if type_perso == "Personnage":
        return Personnage(dico["nom"], dico["points_de_vie"], dico["force"])
    raise ValueError(f"Type de personnage inconnu : {type_perso}")


def charger_personnage(chemin):
    with open(chemin, "r", encoding="utf-8") as f:
        dico = json.load(f)
    return personnage_depuis_dict(dico)
```

5 POURQUOI COMME ÇA

La clé `"type"` permet de choisir la bonne classe : c'est ce qui rend l'aller-retour **sauvegarder → charger** fidèle (le test `lancelot_recharge.mana` renvoie bien 5 : on a retrouvé un vrai `Heros`, pas un simple `Personnage`). `raise ValueError` est le moyen standard de dire « donnée invalide » sans être confondu avec un résultat normal. Isoler `personnage_depuis_dict` évite de maintenir deux fois la même logique.

Partie D — Gérer une partie avec historique

Objectif : une classe Partie qui fait combattre des personnages, garde la trace des événements et se sauvegarde.

D.1 Partie.__init__ et ajouter_personnage(self, personnage)

1 LA CONSIGNE

Le constructeur est donné (deux listes vides : `personnages` et `historique`). « `ajouter_personnage` : ajoute `personnage` à `self.personnages`. »

2 COMPRENDRE LA CONSIGNE

Une `Partie` **contient** des personnages (relation de composition : c'est une autre façon de combiner les classes que l'héritage). Elle n'a pas besoin de savoir si ce sont des héros ou des monstres : une liste suffit.

3 COMMENT S'Y PRENDRE

Une seule étape : `append` sur la liste.

4 COMMENT C'EST RÉALISÉ

Votre code

```
class Partie:
    def __init__(self):
        self.personnages = []
        self.historique = []   # messages, dans l'ordre chronologique

    def ajouter_personnage(self, personnage):
        self.personnages.append(personnage)
```

5 POURQUOI COMME ÇA

`append` ajoute à la fin, ce qui conserve l'ordre d'arrivée. Garder les personnages dans la partie (et non dans des variables isolées) permet ensuite de tous les sauvegarder d'un coup.

D.2 combat(self, attaquant, defenseur)

1 LA CONSIGNE

« Fait attaquer `defenseur` par `attaquant` (réutilisez `attaquer` : c'est pour cela qu'elle doit *retourner* un message), ajoute ce message à `self.historique`, puis, si `defenseur` n'est plus vivant après l'attaque, ajoute aussi un message du type « *Gobelin est vaincu !* ». »

2 COMPRENDRE LA CONSIGNE

La méthode **orchestre** : elle ne calcule pas les dégâts elle-même, elle laisse l'attaquant le faire. Elle **n'affiche rien** : elle archive. Il y a deux messages possibles : toujours celui de l'attaque, et celui de la défaite seulement si le défenseur vient de tomber à 0.

3 COMMENT S'Y PRENDRE

1. Appeler `attaquant.attaquer(defenseur)` et récupérer le message.
2. L'ajouter à `self.historique`.
3. Tester `defenseur.est_vivant()` ; si faux, ajouter le message de défaite.

4 COMMENT C'EST RÉALISÉ

Votre code

```
def combat(self, attaquant, defenseur):
    atq = attaquant.attaquer(defenseur)
    self.historique.append(atq)
    if not defenseur.est_vivant():
        msg_dft = f"{defenseur.nom} est mort"
        self.historique.append(msg_dft)
```

Les trois étapes sont respectées et c'est bien le `return` de `attaquer` qui alimente l'historique : tout le travail de la partie A trouve ici son sens.

> **Point d'attention.** Si le défenseur était **déjà** à 0 PV avant le combat (cas du test : le gobelin est déjà tombé en partie B), `attaquer` répond « déjà vaincu », puis `combat` ajoute *aussi* « est mort » : l'historique annonce une mort qui n'a pas eu lieu pendant ce combat. De plus, la consigne propose le texte « est vaincu ! ».

Version corrigée

```
def combat(self, attaquant, defenseur):
    etait_vivant = defenseur.est_vivant()          # on note l'état AVANT l'attaque
    self.historique.append(attaquant.attaquer(defenseur))
    if etait_vivant and not defenseur.est_vivant():
        self.historique.append(f"{defenseur.nom} est vaincu !")
```

5 POURQUOI COMME ÇA

Délégation + polymorphisme : `combat` appelle `attaquant.attaquer(...)` sans savoir si c'est un `Personnage`, un `Heros` ou un `Monstre` ; chacun applique sa propre règle de dégâts. Ajouter une nouvelle classe demain ne demandera aucune modification de `Partie`. Mémoriser `etait_vivant` avant l'attaque est nécessaire pour distinguer « vient de mourir » de « était déjà mort ».

Sortie de la version corrigée sur une séquence de 4 combats (le dernier vise un gobelin déjà vaincu) :

```
Lancelot attaque Gobelin et lui inflige 10 dégâts (Gobelin : 10 PV restants)
Gobelin attaque Lancelot et lui inflige 10 dégâts (Lancelot : 30 PV restants)
Lancelot attaque Gobelin et lui inflige 10 dégâts (Gobelin : 0 PV restants)
Gobelin est vaincu !
Gobelin est déjà vaincu
```

D.3 sauvegarder(self, chemin) dans Partie

1 LA CONSIGNE

« Écrit dans `chemin`, au format JSON, un dictionnaire `{"historique": self.historique, "personnages": [...]}` où `"personnages"` est la liste des `to_dict()` de chaque personnage. »

2 COMPRENDRE LA CONSIGNE

On assemble **un seul dictionnaire** qui contient tout l'état de la partie. L'historique (liste de textes) est déjà compatible JSON. Les personnages ne le sont pas (ce sont des objets) : il faut les convertir un par un avec `to_dict()`, d'où la liste de dictionnaires.

3 COMMENT S'Y PRENDRE

1. Créer le dictionnaire avec l'historique et une liste vide pour les personnages.
2. Parcourir `self.personnages` et ajouter chaque `to_dict()`.
3. Écrire le tout avec `json.dump` dans un `with open(..., "w")`.

4 COMMENT C'EST RÉALISÉ

Votre code (légèrement remis en forme)

```
def sauvegarder(self, path):
    data = {"historique": self.historique, "personnages": []}
    for perso in self.personnages:
        data["personnages"].append(perso.to_dict())
    with open(path, "a", encoding="utf-8") as file:
        json.dump(data, file, ensure_ascii=False, indent=4)
```

Ici vous utilisez bien `with` (c'est correct). Le plan est suivi à la lettre. Même remarque qu'en C.2 : le mode `"a"` doit devenir `"w"`, sinon la 2e sauvegarde rend le fichier invalide. La boucle peut aussi s'écrire en une ligne (compréhension de liste) :

Version corrigée

```
def sauvegarder(self, chemin):
    data = {
        "historique": self.historique,
        "personnages": [p.to_dict() for p in self.personnages],
    }
    with open(chemin, "w", encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False, indent=4)
```

5 POURQUOI COMME ÇA

`perso.to_dict()` est polymorphe : chaque personnage fournit sa propre description (avec mana pour un héros, butin pour un monstre) sans que `Partie` le sache. Le dictionnaire final ne contient que des types que JSON comprend : c'est ce qui rend l'écriture possible.

D.4 (Bonus) charger_partie(chemin)

1 LA CONSIGNE

« Écrivez une fonction `charger_partie(chemin)` qui relit un fichier de partie et reconstruit une `Partie` complète (personnages + historique), en réutilisant la logique de `charger_personnage`. »

2 COMPRENDRE LA CONSIGNE

Opération inverse de `Partie.sauvegarder` : on retrouve un dictionnaire à deux clés ; l'historique est directement réutilisable ; chaque dictionnaire de personnage doit redevenir un objet de la bonne classe, comme en C.3.

3 COMMENT S'Y PRENDRE

1. Lire le fichier avec `json.load`.
2. Créer une `Partie()` vide.
3. Pour chaque dictionnaire de `data["personnages"]` : reconstruire l'objet et l'ajouter.
4. Copier l'historique, puis retourner la partie.

4 COMMENT C'EST RÉALISÉ

Votre code (constructeurs abrégés avec « ... »)

```
def charger_partie(path):
    file = open(path, "r", encoding="utf-8")
    data = json.load(file)
    partie = Partie()
    for dico in data["personnages"]:
        if dico["type"] == "Heros":
            perso = Heros(dico["nom"], dico["points_de_vie"], dico["force"], dico["mana"])
        elif dico["type"] == "Monstre":
            perso = Monstre(...)          # idem charger_personnage
        elif dico["type"] == "Personnage":
            perso = Personnage(...)
        else:
            return f"Il n'existe pas de type de personnage de la sorte {dico['type']}"
        partie.ajouter_personnage(perso)
    partie.historique = data["historique"]
    return partie
```

Le raisonnement est juste et complet. Mais la consigne demande de **réutiliser** la logique de `charger_personnage` alors que vous l'avez **recopiée** ; les deux défauts de C.3 (fichier non refermé, texte retourné au lieu d'une erreur) s'y retrouvent aussi — avec ici une conséquence pire : en cas d'erreur la fonction retourne un texte à la place d'une `Partie`.

Version corrigée (utilise personnage_depuis_dict de la partie C)

```
def charger_partie(chemin):
    with open(chemin, "r", encoding="utf-8") as f:
        data = json.load(f)
    partie = Partie()
    for dico in data["personnages"]:
        partie.ajouter_personnage(personnage_depuis_dict(dico))
    partie.historique = data["historique"]
    return partie
```

5 POURQUOI COMME ÇA

Une fonction partagée évite la duplication : si un nouveau type de personnage apparaît, **une seule** fonction à modifier. C'est le principe « ne pas se répéter » (DRY), le même qui justifiait `super()` dans les classes.

Bilan : points à corriger et notions à retenir

Ce qui est solide dans votre code, et ce qui demande un ajustement.

Ce qui est solide : l'architecture complète est juste (4 classes, héritage, *super()*, *to_dict* en cascade, polymorphisme exploité dans *sauvegarder* et *combat*), la logique de rechargement par la clé *type* est correcte, et vous avez pris des initiatives pertinentes (garde « déjà vaincu »). Les corrections portent sur des **détails d'exécution**, mais deux d'entre eux changent réellement les résultats (compétence spéciale, sauvegarde en mode append).

Tableau des écarts (tous vérifiés en exécutant le code)

#	Où	Problème constaté	Correction
1	Heros.competence_speciale	Dégâts infligés même sans mana ; test du mana fait après la soustraction (le 2e appel remet le mana à 0 au lieu de 5) ; *print* au lieu de *return*.	Tester **avant** d'agir, retourner le message d'échec.
2	Personnage.sauvegarder Partie.sauvegarder	Mode "a" : la 2e sauvegarde produit un JSON invalide (*Extra data*) ; fichier non refermé (Personnage).	Mode "w" + *with*.
3	Personnage.__repr__	Affiche « Aldric : 20 PV » au lieu de « Aldric (20 PV) ».	f"{nom} ({pv} PV)".
4	attaquer (Personnage, Monstre)	« {degats}dégats » sans espace ; « (Gobelin: ... » sans espace avant « : » ; garde « déjà vaincu » absente dans Monstre.	Corriger le f-string ; même garde dans Monstre.
5	Partie.combat	« est mort » ajouté même si la cible était déjà vaincue ; texte différent de « est vaincu ! ».	Mémoriser *etait_vivant* avant l'attaque.
6	charger_personnage charger_partie	Fichier non refermé ; type inconnu → retourne un texte ; logique dupliquée.	*with*, *raise ValueError*, fonction *personnage_depuis_dict*.
7	Personnage.subir_degats	Dernière ligne sans effet (*x = x*).	La supprimer, ou *max(0, ...)*.

Mémo des notions utilisées

Notion	En une phrase	Où on l'a vue
Constructeur / self	*__init__* crée l'objet ; *self.x* range une donnée dans l'objet.	A.0, B.1, B.3
return vs print	*print* affiche et renvoie None ; *return* donne une valeur réutilisable.	A.2, B.2, D.2
Héritage / super()	La classe fille reprend le parent ; *super()* appelle sa version.	B.1, B.3, C.1
Redéfinition (override)	Même nom de méthode dans la fille, comportement adapté.	B.4, C.1
Polymorphisme	Même appel, comportement choisi selon la classe réelle de l'objet.	B.4, C.2, D.2, D.3
Clause de garde	Vérifier d'abord le cas impossible, retourner tôt, agir ensuite.	A.2, B.2
Sérialisation JSON	Objet → dictionnaire → texte (*json.dump*) ; et l'inverse (*json.load*).	C.1 à C.3, D.3, D.4

Notion	En une phrase	Où on l'a vue
with open(...)	Ouvre un fichier et le referme automatiquement. "w" écrase, "a" ajoute, "r" lit.	C.2, C.3, D.3
Délégation / composition	Un objet (*Partie*) en contient d'autres et leur confie le travail.	D.1, D.2

Annexe — Code complet corrigé

Le fichier entier avec toutes les corrections, qui produit les sorties attendues par l'énoncé.

```
import json

class Personnage:
    """Un personnage générique : nom, points de vie, force d'attaque."""

    def __init__(self, nom, points_de_vie, force):
        self.nom = nom
        self.points_de_vie = points_de_vie
        self.force = force

    def est_vivant(self):
        return self.points_de_vie > 0

    def subir_degats(self, degats):
        self.points_de_vie = max(0, self.points_de_vie - degats)

    def attaquer(self, cible):
        if not cible.est_vivant():
            return f"{cible.nom} est déjà vaincu"
        degats = self.force
        cible.subir_degats(degats)
        return (f"{self.nom} attaque {cible.nom} et lui inflige {degats} dégâts "
                f"({cible.nom} : {cible.points_de_vie} PV restants)")

    def to_dict(self):
        return {
            "type": "Personnage",
            "nom": self.nom,
            "points_de_vie": self.points_de_vie,
            "force": self.force,
        }

    def sauvegarder(self, chemin):
        with open(chemin, "w", encoding="utf-8") as f:
            json.dump(self.to_dict(), f, ensure_ascii=False, indent=4)

    def __repr__(self):
        return f"{self.nom} ({self.points_de_vie} PV)"

class Heros(Personnage):

    def __init__(self, nom, points_de_vie, force, mana):
        super().__init__(nom, points_de_vie, force)
        self.mana = mana

    def competence_speciale(self, cible):
        if self.mana < 10:
            return (f"{self.nom} n'a pas assez de mana "
                    f"({self.mana} < 10) : compétence impossible")
        degats = self.force * 2
        cible.subir_degats(degats)
        self.mana -= 10
        return (f"{self.nom} utilise sa compétence spéciale sur {cible.nom} "
                f"et lui inflige {degats} dégâts "
                f"({cible.nom} : {cible.points_de_vie} PV restants, "
                f"mana de {self.nom} : {self.mana})")

    def to_dict(self):
        dico = super().to_dict()
        dico["type"] = "Heros"
        dico["mana"] = self.mana
        return dico
```

```
class Monstre(Personnage):

    def __init__(self, nom, points_de_vie, force, niveau_de_danger, butin):
        super().__init__(nom, points_de_vie, force)
        self.niveau_de_danger = niveau_de_danger
        self.butin = butin

    def attaquer(self, cible):
        if not cible.est_vivant():
            return f"{cible.nom} est déjà vaincu"
        degats = self.force * self.niveau_de_danger
        cible.subir_degats(degats)
        return (f"{self.nom} attaque {cible.nom} et lui inflige {degats} dégâts "
                f"({cible.nom} : {cible.points_de_vie} PV restants)")

    def to_dict(self):
        dico = super().to_dict()
        dico["type"] = "Monstre"
        dico["niveau_de_danger"] = self.niveau_de_danger
        dico["butin"] = self.butin
        return dico

def personnage_depuis_dict(dico):
    type_perso = dico["type"]
    if type_perso == "Heros":
        return Heros(dico["nom"], dico["points_de_vie"],
                     dico["force"], dico["mana"])
    if type_perso == "Monstre":
        return Monstre(dico["nom"], dico["points_de_vie"], dico["force"],
                       dico["niveau_de_danger"], dico["butin"])
    if type_perso == "Personnage":
        return Personnage(dico["nom"], dico["points_de_vie"], dico["force"])
    raise ValueError(f"Type de personnage inconnu : {type_perso}")

def charger_personnage(chemin):
    with open(chemin, "r", encoding="utf-8") as f:
        dico = json.load(f)
    return personnage_depuis_dict(dico)

class Partie:

    def __init__(self):
        self.personnages = []
        self.historique = []

    def ajouter_personnage(self, personnage):
        self.personnages.append(personnage)

    def combat(self, attaquant, defenseur):
        etait_vivant = defenseur.est_vivant()
        self.historique.append(attaquant.attaquer(defenseur))
        if etait_vivant and not defenseur.est_vivant():
            self.historique.append(f"{defenseur.nom} est vaincu !")

    def sauvegarder(self, chemin):
        data = {
            "historique": self.historique,
            "personnages": [p.to_dict() for p in self.personnages],
        }
        with open(chemin, "w", encoding="utf-8") as f:
            json.dump(data, f, ensure_ascii=False, indent=4)

def charger_partie(chemin):
    with open(chemin, "r", encoding="utf-8") as f:
        data = json.load(f)
    partie = Partie()
    for dico in data["personnages"]:
        partie.ajouter_personnage(personnage_depuis_dict(dico))
    partie.historique = data["historique"]
    return partie
```

Sorties du jeu de tests de l'énoncé (parties A, B et C) avec ce code

```
Aldric attaque Gobelin et lui inflige 8 dégâts (Gobelin : 12 PV restants)
Gobelin (12 PV)
Gobelin attaque Lancelot et lui inflige 10 dégâts (Lancelot : 30 PV restants)
Lancelot utilise sa compétence spéciale sur Gobelin et lui inflige 20 dégâts (... mana : 15)
Lancelot utilise sa compétence spéciale sur Gobelin et lui inflige 20 dégâts (... mana : 5)
Lancelot n'a pas assez de mana (5 < 10) : compétence impossible
Lancelot (30 PV)
5
```